Algorithms 1
Homework 7

Nikita Zagainov

November 11, 2024

Расширенный алгоритм Евклида. Куча. Разные задачи. Графы.

1. Запишем общую форму уравнения:
$$ax + by = c$$
Пусть нам дана пара целых чисел x_0, y_0, удовлетворяющих данному уравнению. Тогда решением уравнения также будут
$$x = x_0 + k \cdot \frac{b}{\gcd(a,b)}, \quad y = y_0 - k \cdot \frac{a}{\gcd(a,b)}, \quad \forall k \in \mathbb{Z}.$$
Для нахождения x_0, y_0 можно воспользоваться расширенным алгоритмом Евклида:
   - Найти x_0', y_0', удовлетворяющие уравнению $ax_0' + by_0' = \gcd(a,b)$.
   - Если $\frac{c}{\gcd(a,b)}$ не является целым числом, то уравнение не имеет решений. В противном случае берем $x_0 = x_0' \cdot \frac{c}{\gcd(a,b)}$ и $y_0 = y_0' \cdot \frac{c}{\gcd(a,b)}$.

   Алгоритм:
```python
def extended_gcd(a, b):
    if b == 0:
        return (a, 1, 0)
    d, x, y = extended_gcd(b, a % b)
    return (d, y, x - (a // b) * y)


def solve(a, b, c):
    d, x_0, y_0 = extended_gcd(a, b)
    if c % d != 0:
        return None
    return (x_0 * (c // d), y_0 * (c // d), b // d, a // d)
```
   Применяя его, получаем:
   1. $x = -399 + 55k, \quad y = 247 - 34k$
   2. Нет решений

2. Задача $ax + b \equiv 0 \pmod{m}$ эквивалентна задаче $ax + my = -b$. Эта задача решается аналогично предыдущей.

   Ответ: $x = 40 + 33k, \quad \forall k \in \mathbb{Z}$.

3. Эта задача может быть переформулирована следующим образом:
$$ax \equiv 1 \pmod{m}$$
Она эквивалентна задаче $ax + my = 1$. По аналогии с первой задачей, решаем задачу $7x + 102y = 1$ и получаем ответ: $x = -29 + 102k, \quad \forall k \in \mathbb{Z}$.

4. Чтобы добавить элемент в кучу, можно сначала добавить его в конец списка, а затем восстановить свойство кучи. Это можно сделать, сравнивая этот элемент с родителем и меняя их местами, если родитель меньше ребенка. Продолжаем этот процесс, пока родитель не станет больше ребенка, или пока не дойдем до корня кучи.

   Алгоритм:

```python
def heappush(heap: list[float], item: float) -> None:
    heap.append(item)

    pos = len(heap) - 1
    while pos > 0:
        parent_pos = (pos - 1) // 2
        if heap[parent_pos] < heap[pos]:
            heap[parent_pos], heap[pos] = heap[pos], heap[parent_pos]
            pos = parent_pos
        else:
            break
```

   Сложность данного алгоритма — $O(\log n)$, так как в худшем случае мы пройдем по высоте дерева, которая равна $\lceil \log_2 n \rceil$.

5. Реализовать очередь с помощью двух стеков можно следующим образом:

   - HEAPIFY сортирует массив за $O(n \log n)$ и поочередно складывает их в один стек.
   - POP берет верхний элемент из стека за $O(1)$.
   - PUSH создаёт временный стек, в который кладёт элементы с верхушки основного стека, пока не дойдёт до элемента, который больше добавляемого. Затем добавляет элемент в основной стек и возвращает элементы из временного стека обратно в основной. Сложность — $O(n)$.
   - PEEK возвращает верхний элемент стека за $O(1)$.

   Реализация:

```python
class MinQueueStack:
    def __init__(self, elems: list[float]):
        self.stack = sorted(elems, reverse=True)

    def push(self, elem: float):
        temp_stack = []
        while len(self) > 0 and self.peek() < elem:
            temp_stack.append(self.pop())
        self.stack.append(elem)
        while len(temp_stack) > 0:
            self.stack.append(temp_stack.pop())

    def pop(self) -> float:
        return self.stack.pop(-1)

    def peek(self) -> float:
        return self.stack[-1]

    def __len__(self) -> int:
        return len(self.stack)
```

6. Количество уникальных лесов обхода у графа пути равно количеству способов выбрать подмножество из всех $|V| - 1$ рёбер, то есть $2^{|V|-1}$.

   Для любого такого подмножества найдётся ровно один лес обхода. Для начала возьмём дерево, включающее в себя конечную вершину начнём обход с корня этого дерева. Далее аналогично обходим оставшиеся деревья.

7. Чтобы найти путь длины $n-1$ в графе, можно использовать DFS от каждой из вершин, пока не найдём искомый путь. Обход графа занимает $O(V+E)$, таких обходов будет V, так что общая сложность алгоритма — $O(V^2+VE)$.

   Алгоритм:

```python
def dfs(graph, node, visited, path):
    visited[node] = True
    path.append(node)
    if len(path) == len(graph):
        return path
    for neighbor in graph[node]:
        if not visited[neighbor]:
            new_path = dfs(graph, neighbor, visited, path)
            if new_path:
                return new_path
    visited[node] = False
    path.pop()
    return None


def find_path(graph: dict):
    visited = {node: False for node in graph}
    for node in graph:
        path = dfs(graph, node, visited, [])
        if path:
            return path
    return None
```

   Доказать, что путь всегда существует, не получилось :(

8.
   - Шаг: 1, вершины: $[a]$
   - Шаг: 2, вершины: $[a, b]$
   - Шаг: 3, вершины: $[a, b, c]$
   - Шаг: 4, вершины: $[a, b, c, f]$
   - Шаг: 5, вершины: $[a, b, c, f, e]$
   - Шаг: 6, вершины: $[a, b, c, f, e, d]$

9. Данная функция возрастает на множестве натуральных чисел, так как все коэффициенты неотрицательны. Поэтому можно воспользоваться бинарным поиском для определения x. Если в процессе был найден подходящий x, то возвращаем $True$, иначе $False$, так как решение будет нецелым. Для этого нужно найти границы для бинарного поиска. Это можно сделать путём итеративного экспоненциального увеличения границ, пока максимум не превзойдёт y.

   Алгоритм:

```python
def p(a: list[int], x: int) -> int:
    result = 0
    power = 1
    for i in range(len(a)):
        result += a[i] * power
        power *= x

    return result


def solve(a: list[int], y: int) -> bool:
    if y == a[0]:
        return True
    if y < a[0]:
        return False

    x_min = 0
```

```
    x_max = 1

    while p(a, x_max) < y:
        x_min = x_max
        x_max *= 2

    while x_min <= x_max:
        x = (x_min + x_max) // 2
        value = p(a, x)
        if value == y:
            return True
        if value < y:
            x_min = x + 1
        else:
            x_max = x - 1

    return False
```

Заметим, что $x = O((\frac{y}{a_n})^{\frac{1}{n}})$, Тогда сложность алгоритма:

$$O\left(n(\frac{y}{a_n})^{\frac{1}{n}} + n\log\left((\frac{y}{a_n})^{\frac{1}{n}}\right)\right)$$

10. Данную задачу можно решить следующим образом:

    - Разбить все этажи на несколько отрезков
    - С последнего этажа каждого отрезка сбрасывать шарик, пока он не разобьется. Если уцелел, то переходим к следующему отрезку.
    - Проходим все этажи этого отрезка снизу вверх, пока второй шарик не разобьется.
    - ответом будет последний этаж, с которого шарик не разбился.

    Если количество этажей на отрезке, $k = const$, то сложность алгоритма будет $O(k + \frac{n}{k})$, где n — количество этажей. Это выражение минимально при $k = \sqrt{n}$:

    $$O(\sqrt{n} + \sqrt{n}) = O(\sqrt{n})$$

    Для более оптимального числа бросков, можно изменять k в зависимости от количества оставшихся этажей, выбирая $k = \lceil\sqrt{n}\rceil$, где n — количество оставшихся этажей. Тогда в худшем случае, когда прочность будет эквивалентна последнему этажу на каком-либо отрезке, надо будет сделать не более 17 бросков. Выбор $k = \lceil\sqrt{(n)}\rceil$ разобьёт этажи на следующие отрезки:

    $$\begin{aligned}&[1..10], [11..20], [21..29], [30..38], [39..46],\\&[47..54], [55..61], [62..68], [69..74], [75..80],\\&[81..85], [86..89], [90..93], [94..96], [97..98],\\&[99..100]\end{aligned}$$

    Для каждого из этих отрезков сумма количества предшествующих отрезков и длины не превышает 18, но в во время второго прохода можно не кидать шарик с последнего этажа отрезка, так что количество бросков не превышает 17.